



**School of Computer**

**Science and**

**Engineering**

**Winter Semester 2022-2023**

**Continuous Assessment Test – 2**

SLOT: A1

**Programme Name & Branch:** B.Tech – CSE ( BCI, BCE, BCT, BDS, BCB)

**Course Name & code:** Data Structures and Algorithms – BCSE202L

**Class Number (s):** VL2022230505842, 6331, 5847, 5851, 5837, 6304, 5855, 5849, 5840

**Faculty Name (s):** Joshva Devadas T, Priyanka N, Sayan Sikdar, Naveenkumar J, Akash Sinha, Kalaivani K, Ganesh Shamrao Khokare, Sunil Kumar PV, S M Farooq.

**Exam Duration: 90 Min.**

**Maximum Marks: 50**

**General instruction(s):**

All Questions carry Equal Marks.

| Q.No.       | Question                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | Max Marks |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| 1.          | <p><b>a)</b> You are given a Singly Linked list below.<br/>Head <math>\rightarrow A \rightarrow B \rightarrow A \rightarrow B \rightarrow A \rightarrow B \rightarrow \text{NULL}</math><br/>Write the procedure to perform an alternating split of the given singly linked list. The expected output after splitting the singly linked list should result in two singly linked lists each containing only similar characters.<br/>(4 Marks)</p> <p><b>b)</b> Write a procedure to check if two circular linked lists are identical. What will be the time complexity of this procedure. Illustrate the working of the procedure using the given circular linked list.<br/>List 1: 1 <math>\rightarrow</math> 2 <math>\rightarrow</math> 3 <math>\rightarrow</math> 1<br/>List 2: 1 <math>\rightarrow</math> 2 <math>\rightarrow</math> 3 <math>\rightarrow</math> 1<br/>(6 Marks)</p> | 10        |
| <b>Sol.</b> | <p><b>a) Procedure: 3 Marks</b></p> <ol style="list-style-type: none"><li>1. Initialize two new linked lists, listA and listB, to store the resulting split lists.</li><li>2. Initialize a pointer current to the head of the input linked list.</li><li>3. While current is not NULL:<ol style="list-style-type: none"><li>a. Append the current node to listA.</li><li>b. Move current to its next node.</li><li>c. If current is not NULL:<ol style="list-style-type: none"><li>i. Append the current node to listB.</li><li>ii. Move current to its next node.</li></ol></li></ol></li><li>4. Set the next pointers of the last nodes in listA and listB to NULL.</li></ol> <p>After performing this procedure on the given singly linked list, we get two</p>                                                                                                                     |           |

|      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |    |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
|      | <p><b>new singly linked lists : 1 Mark</b></p> <p>listA: Head -&gt; A -&gt; A -&gt; A -&gt; NULL</p> <p>listB: Head -&gt; B -&gt; B -&gt; B -&gt; NULL</p> <p><b>b) Procedure: 3 Marks</b></p> <ol style="list-style-type: none"> <li>1. Initialize two pointers, one for each list. Let's call them ptr1 and ptr2. Set ptr1 to point to the first node (START1) of List 1 and ptr2 to point to the first node (START2) of List 2.</li> <li>2. While ptr1 is not equal to the first node of List 1 or ptr2 is not equal to the first node of List 2: <ol style="list-style-type: none"> <li>a. Compare the data of the current nodes pointed to by ptr1 and ptr2.</li> <li>b. If the data is not the same, return false.</li> <li>c. Move both pointers PTR1 and PTR2 to their next nodes.</li> </ol> </li> <li>3. If both pointers reach their starting points (START1 and START2) at the same time, return true. Otherwise, return false.</li> </ol> <p><b>Complexity: 1 Mark</b></p> <p>The time complexity of this procedure is <math>O(n)</math> where <math>n</math> is the number of nodes in the larger list.</p> <p><b>Illustration: 2 Marks</b></p> <ul style="list-style-type: none"> <li>• In this illustration, the data of the first nodes pointed to by ptr1 and ptr2 is the same (both are 1), <ul style="list-style-type: none"> <li>◦ so move both pointers to their next nodes.</li> </ul> </li> <li>• In the next iteration of the loop, the data of the current nodes pointed to by ptr1 and ptr2 is again the same (both are 2), <ul style="list-style-type: none"> <li>◦ so move both pointers to their next nodes.</li> </ul> </li> <li>• In the third iteration of the loop, the data of the current nodes pointed to by ptr1 and ptr2 is again the same (both are 3), <ul style="list-style-type: none"> <li>◦ so move both pointers to their next nodes.</li> </ul> </li> <li>• At this point, both pointers have reached their starting points (the first node of their respective lists), so exit the loop and return true since both lists are identical.</li> </ul> |    |
| 2.   | <p>Suppose you are given an array of strings that need to be sorted in lexicographic order (i.e., dictionary order). How can you modify the quicksort algorithm to handle this requirement? Illustrate the working of your modified quicksort algorithm on the following array of names.</p> <p>names = ["Smith", "Johnson", "Williams", "Brown", "Jones", "Garcia", "Miller", "Davis"]</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | 10 |
| Sol. | <p><b>Modified Quick Sort Algorithm Idea: 2 Marks</b></p> <p>Quicksort can be easily modified to sort an array of strings in lexicographic order. The only change required is in the comparison function used to compare two elements of the array. Instead of using the</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |    |

standard comparison operators (e.g., <, >), need to use a string comparison function such as strcmp in C

### Modified Quick Sort Algorithm/Pseudocode: 5 Marks

// Function to swap two strings in an array

```
void swap(char* arr[], int i, int j) {  
    char* temp = arr[i];  
    arr[i] = arr[j];  
    arr[j] = temp;  
}
```

// Partition function for quicksort

```
int partition(char* arr[], int low, int high) {  
    char* pivot = arr[high];  
    int i = low - 1;  
    for (int j = low; j < high; j++) {  
        if (strcmp(arr[j], pivot) < 0) {  
            i++;  
            swap(arr, i, j);  
        }  
    }  
    swap(arr, i+1, high);  
    return i+1;  
}
```

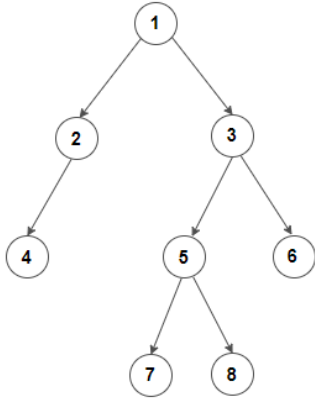
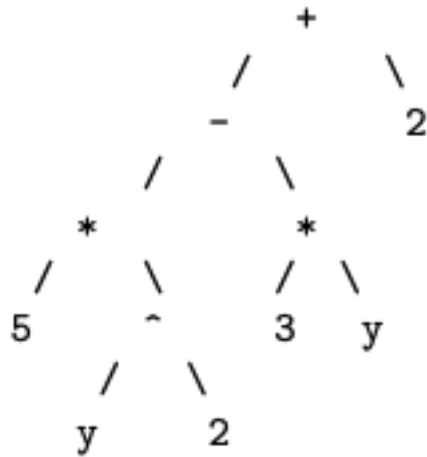
// Modified quicksort function for lexicographic sorting of an array of strings

```
void quicksort_lex(char* arr[], int low, int high) {  
    if (low < high) {  
        int pi = partition(arr, low, high);  
        quicksort_lex(arr, low, pi-1);  
        quicksort_lex(arr, pi+1, high);  
    }  
}
```

### Illustration Steps: 3 Marks

- Choose a pivot element, say “Johnson”.
- Partition the array into two sub-arrays: elements that are lexicographically less than “Johnson” and elements that are lexicographically greater than “Johnson”. After partitioning, the array becomes [“Garcia”, “Davis”, “Brown”, “Johnson”, “Jones”, “Miller”, “Smith”, “Williams”].
- Recursively apply the modified quicksort algorithm to the two sub-arrays.

```
char* arr[] = {"Smith", "Johnson", "Williams", "Brown", "Jones",  
"Garcia", "Miller", "Davis"};  
int n = sizeof(arr) / sizeof(arr[0]);
```

|      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |    |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
|      | quicksort_lex(arr, 0, n-1);                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |    |
| 3.   | <p><b>a)</b> Construct the Binary tree from given Traversals. (4 Marks)</p> <p>Inorder Traversal : { 4, 2, 1, 7, 5, 8, 3, 6 }</p> <p>Postorder Traversal: { 4, 2, 7, 8, 5, 6, 3, 1 }</p> <p><b>b)</b> Build a expression tree from the given expression: (6 Marks)</p> <p style="text-align: center;"><math>((5 * (y^2)) - (3 * y)) + 2,</math></p> <p>Also using the built expression tree extract the Postfix and prefix Expressions.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 10 |
| Sol. | <p><b>a) Binary tree – 4 Marks.</b></p>  <pre> graph TD     1((1)) --&gt; 2((2))     1 --&gt; 3((3))     2 --&gt; 4((4))     3 --&gt; 5((5))     3 --&gt; 6((6))     5 --&gt; 7((7))     5 --&gt; 8((8)) </pre> <p><b>b) Expression Tree – 4 Marks.</b></p>  <pre> graph TD     Plus["+"] --&gt; Minus["-"]     Plus --&gt; 2["2"]     Minus --&gt; Star1["*"]     Minus --&gt; Star2["*"]     Star1 --&gt; 5["5"]     Star1 --&gt; Caret["^"]     Caret --&gt; y1["y"]     Caret --&gt; 2_["2"]     Star2 --&gt; 3["3"]     Star2 --&gt; y2["y"] </pre> <p>Postfix - 5 y 2 ^ * 3 y * - 2 + <span style="color: red;">1 Marks</span></p> <p>Prefix - + - * 5 ^ y 2 * 3 y 2 <span style="color: red;">1 Marks</span></p> |    |
| 4.   | <p><b>a)</b> Suppose you have a binary search tree containing the ages of all the employees in a company. The tree is balanced and has n nodes. You need to write an algorithm/pseudocode to find the 3rd youngest employee in the company. Illustrate your algorithm's working using the following binary search tree containing the ages of the employees.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | 10 |

|      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |  |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
|      | <p>What is the time complexity of your algorithm for finding the 3rd youngest employee in the company.</p> <pre>       40      /  \     30   50    /  \ /  \   20 35 45 55 </pre> <p>(5 Marks)</p> <p>b) Create an algorithm and write pseudocode to determine if a given binary tree is also a binary search tree. The algorithm should have a time complexity of <math>O(n)</math> and a space complexity of <math>O(n)</math>.<br/>(5 Marks)</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |  |
| Sol. | <p>a) algorithm to find the kth smallest element in a binary search tree.</p> <p><b>Algorithm or Pseudocode – 2 Marks</b></p> <ul style="list-style-type: none"> <li>Perform an inorder traversal of the binary search tree. This will visit the nodes of the tree in ascending order of their values.</li> <li>Keep track of a counter variable as you traverse the tree. Increment the counter each time you visit a node.</li> <li>When the counter reaches k, stop the traversal, and return the value of the current node.</li> </ul> <pre> function kthSmallest(root, k)     stack = empty stack     node = root     while (node is not null or stack is not empty)         while (node is not null)             push node onto stack             node = node.left         node = pop stack         k = k - 1         if k == 0             return node.value         node = node.right </pre> <p><b>Illustration – 2 Marks</b></p> <ul style="list-style-type: none"> <li>Let's say we want to find the 3rd youngest employee in the company (i.e., <math>k = 3</math>).</li> <li>start by performing an inorder traversal of the binary search tree. This visits the nodes in ascending order: 20, 30, 35, 40, 45, 50, 55.</li> <li>keep track of a counter variable as traversing the tree. The counter starts at 0 and is incremented each time we visit a node.</li> <li>When we visit the first node (20), the counter becomes 1. When we visit the second node (30), the counter becomes 2. When we visit the third node (35), the counter becomes 3.</li> </ul> |  |

- Since  $k = 3$ , stop the traversal at this point and return the value of the current node (35).
- Therefore, the age of the 3rd youngest employee in the company is 35.

#### Time Complexity – 1 Mark

The time complexity of the algorithm to find the  $k$ -th smallest element in a binary search tree (BST) is  $O(h + k)$ , where  $h$  is the height of the BST and  $k$  is the rank of the element that is to be found.

The algorithm performs an in-order traversal of the BST using a stack. Each node is visited once during the traversal, and the time it takes to visit a node is  $O(1)$ . So, the total time it takes to visit all nodes is  $O(n)$ , where  $n$  is the number of nodes in the BST.

However, we don't need to visit all nodes in the BST. Required to only visit the first  $k$  nodes in the in-order traversal. So, the time complexity of the algorithm is  $O(h + k)$ , where  $h$  is the height of the BST and  $k$  is the rank of the element needed to be found.

b) A better solution is to look at each node only once. The trick is to write a utility helper function `IsBSTUtil(struct BinaryTreeNode* root, int min, int max)` that traverses down the tree keeping track of the narrowing min and max allowed values as it goes, looking at each node only once. The initial values for min and max should be `INT_MIN` and `INT_MAX` – they narrow from there. (2 Marks)

#### Pseudocode: 3 Marks

```
Initial call: IsBST(root, INT_MIN, INT_MAX);
int IsBST(struct BinaryTreeNode *root, int min, int max) {
    if(!root)
        return 1;
    return (root->data > min && root->data < max &&
            IsBSTUtil(root->left, min, root->data) &&
            IsBSTUtil(root->right, root->data, max));
}
```

OR

By using inorder traversal. The idea behind this solution is that inorder traversal of BST produces sorted lists. While traversing the BST in inorder, at each node check the condition that its key value should be greater than the key value of its previous visited node. Also, we need to initialize the prev with possible minimum integer value (say, `INT_MIN`).

(2 Marks)

#### Pseudocode: 3 Marks

|      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |    |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
|      | <pre> int prev = INT_MIN; int IsBST(struct BinaryTreeNode *root, int *prev) {     if(!root) return 1;      if(!IsBST(root-&gt;left, prev))         return 0;      if(root-&gt;data &lt; *prev)         return 0;      *prev = root-&gt;data;     return IsBST(root-&gt;right, prev); } </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |    |
| 5.   | <p>Assume you are given an array of integers representing the ages of people in a group: [25, 32, 40, 19, 60, 55, 22]. You need to sort this array in ascending order using heap sort with a min heap. Additionally, after sorting the array, two new people join the group with ages 35 and 18. You need to insert these two new ages into the sorted array while maintaining the sorted order.</p> <p>i) How would you perform heap sort on the given array using a min heap? Illustrate and write the pseudocode.</p> <p>ii) How would you insert the two new ages into the sorted array while maintaining the sorted order? Illustrate and write the pseudocode.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | 10 |
| Sol. | <p>i) <b>Procedure: 1 Mark</b></p> <ol style="list-style-type: none"> <li>1. Build a min heap from the input array. It can be done by starting from the last non-leaf node and moving upwards, calling the min-heapify function on each node. The min-heapify function ensures that the subtree rooted at the current node is a min heap by recursively moving the smallest element to the root.</li> <li>2. Repeatedly extract the minimum element from the min heap and move it to the end of the array. It can be done by swapping the first element (the minimum element) with the last element in the heap and then reducing the size of the heap by one. After each extraction, call the min-heapify function on the root to restore the min heap property.</li> </ol> <p><b>Pseudocode: 2 Marks</b></p> <pre> void buildMinHeap(int arr[], int n) {     for (int i = n/2 - 1; i &gt;= 0; i--) {         minHeapify(arr, n, i);     } }  void minHeapify(int arr[], int n, int i) {     int left = 2*i + 1;     int right = 2*i + 2;     int smallest = i;     if (left &lt; n &amp;&amp; arr[left] &lt; arr[smallest]) {         smallest = left;     }     if (right &lt; n &amp;&amp; arr[right] &lt; arr[smallest]) {         smallest = right;     }     if (smallest != i) {         swap(arr[i], arr[smallest]);         minHeapify(arr, n, smallest);     } } </pre> |    |

```

    }
    if (right < n && arr[right] < arr[smallest]) {
        smallest = right;
    }
    if (smallest != i) {
        swap(arr[i], arr[smallest]);
        minHeapify(arr, n, smallest);
    }
}

```

```

void heapsort(int arr[], int n) {
    buildMinHeap(arr, n);
    for (int i = n-1; i >= 0; i--) {
        swap(arr[0], arr[i]);
        minHeapify(arr, i, 0);
    }
}

```

### Illustration: 3 Marks

Given elements are [25, 32, 40, 19, 60, 55, 22].

I1---- 25,32,40,19,60,55,22 => 25,32,22,19,60,55,40

I2---- 25,32,22,19,60,55,40 => 25,19,22,32,60,55,40

I3---- 25,19,22,32,60,55,40 => 19,25,22,32,60,55,40 (Min heap)

### Heap Sort

19,25,22,32,60,55,40 --- swap --- 40,25,22,32,60,55,19 **extract 19**

40,25,22,32,60,55 --- Heapify --- 22, 25, 40, 32, 60, 55

22, 25, 40, 32, 60, 55 --- swap --- 55, 25, 40, 32, 60, 22 **extract 22**

55, 25, 40, 32, 60 --- Heapify --- 25, 32, 40, 55, 60

25, 32, 40, 55, 60 --- swap --- 60, 32, 40, 55, 25 **extract 25**

60,32,40,55 --- Heapify --- 32, 55, 40, 60

32, 55, 40, 60 --- Swap --- 60, 55, 40, 32 **extract 32**

60, 55, 40 --- heapify --- 40, 55, 60

40, 55, 60 --- swap --- 60, 55, 40 **extract 40**

60, 55 --- heapify --- 55, 60

55, 60 --- swap --- 60, 55 **extract 55**

60 – extract 60

Sorted Array- 19, 22, 25, 32, 40, 55, 60

OR



|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |  |
|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
|  | <p>Insert the new elements 35 and 18 into the min heap. The min heap would look like this after inserting 35:</p> <pre>       19      /  \     22   25    /\   /\   32 40 55 60  / 35 </pre> <p>After inserting element 18:</p> <pre>       18      /  \     19   25    /\   /\   22 40 55 60  /\ 32 35 </pre> <p>Extract the minimum element from the min heap and insert it at the end of the sorted array until the heap is empty. The resulting sorted array would be: [18,19,22,25,32,35,40,55,60].</p> |  |
|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|